DIAGRAM SPECIFICATIONS

> **What this file is.** The consolidated register of every diagram the three chapters require. **No diagram is drawn here, deliberately:** a diagram invented for a report rather than derived from the implementation misrepresents the system. Each row fixes the nine attributes the academic guide asks for — number and title, purpose, actors/entities, relationships, flow direction, boundaries, labels and decision points, implementation reference, and how it is to be drawn — and points to the prose specification that carries the detail.
>
> Repository `student-beneficiary-system`, branch `fix/sbms-completion-work`, commit `d6b5032`.

---

1. How to read the register

Column	Meaning
No. / Title	Figure number as used in the chapters; do not renumber
Type	UML or structured notation to draw in
Elements	Actors, classes, tables or components that must appear
Flow / relationships	The connections a reader must be able to follow
Boundary	What the figure must show as inside or outside the system
Decision points / labels	Branches and edge labels that must be visible
Implementation reference	Where the drawn content is proved by code
Detail	Section holding the full prose specification

Status of every figure: SPECIFIED, NOT DRAWN. Twelve of them (4.13–4.24) are screenshots and cannot be produced from specifications at all; they are marked SCREENSHOT REQUIRED and tracked as gaps G-03/G-04.

---

2. Analysis diagrams (Chapter 4 §4.3)

No. / Title	Type	Elements	Flow / relationships	Boundary	Decision points	Implementation reference	Detail
Fig 4.1 System context	Context (Yourdon)	SBMS process; Field Officer, Administrator, Student/guardian, Public verifier; e-mail, Azure hosting, PostgreSQL	Data and control crossing the boundary both ways	SBMS as a single rounded process	—	`backend/src/routes/`, `docs/deployment.md`	§4.3.1
Fig 4.2 Use case	UML use case	3 actors; 14 use cases; `<<include>>` screening	Actor to use case; include arrows	Actors outside a named rectangle	Screening include on registration	route inventory	§4.3.2
Fig 4.3 Activity: registration and screening	UML activity, 4 swimlanes	Officer, API, ML service, Database	Validate → persist → screen → flag → confirm	Swimlanes per tier	Validation failure; ML unavailable; duplicate above cut-off	`students.controller.js`, `mlDuplicateReview.js`	§4.3.3
Fig 4.4 Activity: review, approval, payment	UML activity, 3 swimlanes	Administrator, Field Officer, Database	Queue → reveal → decide → approve → verify → pay → reconcile	Per-tier swimlanes	Reject; duplicate-confirmed; verification rejected; discrepancy	`duplicateFlags.controller.js`, `bursaryAwards.controller.js`, `disbursements.controller.js`	§4.3.4

No. / Title	Type	Elements	Flow / relationships	Boundary	Decision points	Implementation reference	Detail
Fig 4.5 Class	UML class	13 classes incl. `Student`, `DuplicateFlag`, `BursaryAward`, `Disbursement`, `BursaryCertificate`, `AuditLog`	Associations with multiplicities; state enums as attributes	—	—	`backend/src/db/schema.sql`, `backend/src/db/migrations/`	§4.3.5
Fig 4.6 Sequence: registration	UML sequence	Officer browser, API, validator, ML client, Python model, DB	Synchronous calls; ML call inside the request	API as mediator	ML timeout path	`students.controller.js`, `mlDuplicateReview.js`	§4.3.6
Fig 4.7 Sequence: duplicate decision	UML sequence	Administrator, API, DB, audit	Fetch queue → reveal (audited) → decide → write terminal state	—	Confirm / dismiss	`duplicateFlags.controller.js`	§4.3.7
Fig 4.8 Sequence: approval to certificate	UML sequence	Field Officer, Administrator, API, PDF generator, storage	Approve → verify → pay → issue certificate → revoke path	—	`not_current` outcome	`certificates.controller.js`	§4.3.8
Fig 4.9 Sequence: public verification	UML sequence	Phone browser, API, limiter, DB	Token in fragment → API lookup → existence-only response	No authenticated session	Token invalid; rate limited; revoked	`verification.controller.js`	§4.3.9
Fig 4.10 Entity-relationship	Chen	13 entities; key attributes; relationships	1:N, 1:1, M:N resolutions; FK actions	—	—	migrations; schema doc §2–§9	§4.3.10
Fig 4.11 ML pipeline	Two-lane flow	Offline: data → features → split → train → artifact. Online: request → features → score → flag → human	Left-to-right per lane; artifact bridging the lanes	Model boundary vs application	Cut-off 0.50; ML unavailable	`ml-service/`, `train_*.py`	§4.3.11
Fig 4.12 System architecture	Tiered block diagram	Browser SPA; App Service API; PostgreSQL; ML service; file storage	HTTPS between tiers; subprocess or HTTP to model	Trust boundaries drawn	—	`docker-compose.yml`, deployment guide §3	§4.4.1

3. Screen figures (Chapter 4 §4.5) — screenshot required

Each of these is a capture of the running application against **synthetic data only**. The specification states the route, audience, data requested, controls and states, so the capture can be checked rather than posed.

No. / Title	Route	Must show	Implementation reference	Detail
Fig 4.13 Role-specific sign-in	`/login`, `/admin/login`, `/field-officer/login`	SBMS identity block, e-mail and password fields, role switch, error state	`frontend/src/pages/Login.jsx`	§4.5.1
Fig 4.14 Registry list (field officer)	`/students`	Header, role badge, search, table of synthetic students, pagination	`Students.jsx`	§4.5.2
Fig 4.15 Eligibility assessment	`/students/:id/eligibility`	Five inputs, computed category, persisted result	`Eligibility.jsx`	§4.5.2
Fig 4.16 Registration form	`/students/new`	Two-column name fields, identifier fields, five-school selector, validation messages	`StudentRegistration.jsx`	§4.5.3

No. / Title	Route	Must show	Implementation reference	Detail
Fig 4.17 Registration confirmation with screening outcome	post-register view	QR code, generated identifier, screening outcome banner	`StudentRegistration.jsx`, `students.controller.js`	§4.5.3
Fig 4.18 Administrator dashboard	`/dashboard`	The counters the endpoint returns, trend, recent activity	`AdminDashboard.jsx`, `dashboard` route	§4.5.4
Fig 4.19 Duplicate review queue	`/duplicate-review`	Two records per case, masked identifiers, score, reveal control, decision buttons	`DuplicateReview.jsx`	§4.5.5
Fig 4.20 Awards and disbursements	`/disbursements`	Awards table, verification state, payment state, exception panel, certificate action	`Disbursements.jsx`	§4.5.6
Fig 4.21 Administrative support screens	`/reports`, `/audit`, `/users`, `/settings`	Four panels: date-range export, audit rows, user administration, three cut-offs	`Reports.jsx`, `AuditLogs.jsx`, `UserManagement.jsx`, `Settings.jsx`	§4.5.7
Fig 4.22 Student-record verification	`/verify`	Minimal layout, single input, existence-only result	`Verification.jsx`	§4.5.8
Fig 4.23 Certificate verification	`/verify-certificate`	Four outcome states: verified, revoked, not current, unknown	`CertificateVerification.jsx`	§4.5.8
Fig 4.24 Physical deployment architecture	—	Azure resources, network path, TLS, database, storage, ML runtime	`SBMS_DEPLOYMENT_AND_OPERATIONS_GUIDE.md` §3–§7	§4.7

4. Drawing rules that keep the figures honest

1. **Derive, do not decorate.** Every element must be traceable to a file, route or migration listed in the *Implementation reference* column. If an element cannot be traced, it does not belong in the figure.
2. **Draw the human gate.** Figures 4.3, 4.4, 4.7, 4.11 and 4.12 must show the administrator's decision as a step and as a boundary. A diagram that runs model output straight into a state change misrepresents the design.
3. **Keep identifiers as they are.** `field_officer`, `duplicate_flags`, `bursary_certificates` — the diagram must use the code's names, or the figure will not match Chapters Four and Five.
4. **Show failure paths.** ML unavailable, validation failure, rate limit and revoked token are part of the design; omitting them makes the system look more certain than it is.
5. **Captions state provenance.** Screenshots carry the instance (Azure or local), the date and the fact that the data is synthetic. A caption that omits this invites the reader to assume production data.
6. **No invented actors.** There are three roles and one public verifier. A fourth actor in a figure would need a fourth role in the code, and there is none.
7. **One numbering scheme.** Figures are 4.1…4.24 as fixed above; do not renumber when assembling the document, and do not reuse a number for a different figure.

5. Coverage of the guide's diagram expectations

Guide expectation	Figures	Status
Context diagram	4.1	Specified
Use case diagram and use case descriptions	4.2 plus §4.3.2 tables	Specified
Activity diagrams for principal workflows	4.3, 4.4	Specified
Sequence diagrams	4.6, 4.7, 4.8, 4.9	Specified
Class diagram	4.5	Specified
Entity-relationship diagram	4.10	Specified
Architecture diagram	4.12	Specified

Guide expectation	Figures	Status
Deployment diagram	4.24	Specified
System interface / screen designs	4.13–4.23	**Screenshots required** (G-04)
ML pipeline (project-specific)	4.11	Specified

---

6. Related files

`CHAPTER_4_SYSTEM_ANALYSIS_AND_DESIGN.md` §4.3, §4.5, §4.7 (full prose specifications), `FIGURE_PLACEHOLDER_CHECKLIST.md` (capture checklist and pre-insertion checks), `SBMS_UI_SCREEN_AND_WORKFLOW_INVENTORY.md` (screens and workflows behind the figures).